התחיל ב:	28/10/2023, 23:36
מצב	הסתיים
הושלם ב-	28/10/2023, 23:36
הזמן שלקח	8 שניות

שאלה **1**

לא נענה

ניקוד השאלה: 4.00

רשמו את **כל** הפלטים האפשריים של הפונקציה **func**, עבור קלטים שלא גורמים להעלאת שגיאות שלא נתפסות (uncaught exception). יש להניח כי הקלט הוא של טיפוסים סטנדרטים בלבד:

```
def func(lst):
    try:
        print("A",end="")
        lst[0] / lst[1]
        print("B", end="")
    except TypeError as e:
        print("C", end="")
        try:
            lst[0] * lst[1]
            print("D", end="")
            lst[0] + lst[1]
        except IndexError as e:
            print("E", end="")
        except:
            print("F",end="")
        finally:
            print("G", end="")
    finally:
        print("H", end="")
```

רשמו את כל הפלטים האפשרים, מופרדים על ידי פסיק (",") ללא תוספות מיותרות.
רק תשובות בפורמט הנ"ל יבדקו.

שאלה **2**

לא נענה

ניקוד השאלה: 4.00

השלימו את הקוד במקומות המסומנים, כך ששתי הפונקציות (שאינן רקורסיביות) יחזירו את התחילית המשותפת הארוכה ביותר עבור רשימת מילים לא ריקה. אין להשתמש ב comprehensions.

```
def longest_common_prefix1(strings):
    curr = strings[0]
    for word in strings:
        curr = curr[:len(word)]
        while ______(1)______:
            curr = ______(2)______
    return curr

def longest_common_prefix2(strings):
    for i in range(len(strings[0]),-1,-1):
        prefixes = {word[:i] for word in strings}
        if ______(3)______:
            return ______(4)______

strings = ["flower", "flour", "flourish"]
assert longest_common_prefix1(strings) == 'flo'
assert longest_common_prefix2(strings) == 'flo'
```

(1)

(2)

(3)

(4)

שאלה **3**

לא נענה

ניקוד השאלה: 4.00

בלולאה הפנימית, t מתחיל ב-1, ולא ב-0.

נתונה התוכנית הבאה:

```
def f(lst):
    x = 0
    for t in range(1,len(lst)):
        while t < len(lst):
            x += lst[t]
            t*=3
```

מהי סיבוכיות זמן הריצה של הקוד כפונקציה של **n**, כאשר **n** הוא אורך הרשימה **lst**?

שאלה **4**

לא נענה

ניקוד השאלה: 4.00

נתונה התוכנית הבאה:

```
def f(n):
    s = (n / 2) * (n / 3) * (n / 4)
    for i in range(int(s * math.log(n ** 2))):
        if i < math.log(n):
            print(n)
        else:
            break
        for j in range(2 * n):
            for t in range(j):
                print(n)
```

מהי סיבוכיות זמן הריצה של הקוד כפונקציה של **n** חיובי כלשהו?

שאלה **5**

לא נענה

ניקוד השאלה: 4.00

השלימו את הקוד במקום המסומן, כך שהדקורטור **apply_and_sum** יאפשר לפונקציות לפעול על מספר ארגומנטים במקום ארגומנט אחד, והתוצאה תהיה הסכום של הפעלת הפונקציה המקורית על כל ארגומנט.

```
apply_and_sum = ______(1)______

@apply_and_sum
def double(val):
   return 2*val

@apply_and_sum
def square(val):
   return val*val

assert double(5,6)==22      # 2*5+2*6      == 22
assert double(50)==100      # 2*50         == 100
assert square(10,10)==200   # 10*10+10*10 == 200
```

(1)

שאלה **6**

לא נענה

ניקוד השאלה: 4.00

כמה קריאות לפונקציה **g** יגרמו כתוצאה מהקריאה **f(g,x,y,n)** של הקוד הבא כפונקציה של **x**, של **y** ושל **n**?

```
def f(g, x, y, n):
    if n<=0:
        return 0
    else:
        return g(x,y) + max(f(g,x+1,y,n-1), f(g,x,y+1,n-1))
```

יש להניח כי **g** היא פונקציה לא רקורסיבית שמקבלת שני מספרים שלמים כארגומנטים ומחזירה מספר שלם, והקלטים האחרים הם מספרים שלמים ואי שליליים.

רשמו ביטוי מדויק:

שאלה **7**

לא נענה

ניקוד השאלה: 4.00

השלימו את הקוד במקומות המסומנים כך שהפונקציה **high_low(n)** מקבלת מספר שלם חיובי **n** ומחזירה **True** אם הספרות של **n** בסדר מונוטוני יורד משמאל לימין, ו-**False** אחרת. אין להמיר את המספר למחרוזת.

הספרות של מספר הן בסדר מונוטוני יורד אם כל ספרה היא לא יותר קטנה מהספרה שלאחריה. (ראו דוגמאות לאחר הקוד להשלמה).

```
def high_low(n):
    if ______(1)______:
        return True
    elif ______(2)______:
        return False
    else:
        return ______(3)______

assert high_low(2) is True
assert high_low(70) is True
assert high_low(111) is True
assert high_low(112) is False
assert high_low(7674) is False
```

(1)

(2)

(3)

שאלה **8**

לא נענה

ניקוד השאלה: 4.00

מה עומק הרקורסיה המקסימלי של הקריאה **f(x, y)** בקוד הבא כפונקציה של **x** ושל **y**?

```
def f(x, y):
    if x==0 or y==0:
        return 1
    else:
        return f(x-1, y) + f(x, y-1)
```

יש לספור גם את הקריאה הראשונה. יש להניח כי שני הקלטים הם שלמים וחיוביים.

רשמו ביטוי מדויק:

שאלה **9**

לא נענה

ניקוד השאלה: 4.00

ספקו קלטים עבורם הפונקציות הבאות יחזירו **True**. (השלימו את הקוד במקומות המסומנים).

```
def f1(vals):
    index_map = {}
    for i, num in enumerate(vals):
        if num in index_map and index_map[num] == num:
            return True
        index_map[num] = i
    return False

val = ______(1)______
assert f1(val) is True

def f2(n):
    count = 0
    for i in range(1,n+1):
        if n%i==0:
            count+=1
    return count%2==1 and count>3

n = ______(2)______
assert f2(n) is True
```

(1)

(2)

שאלה **10**

לא נענה

ניקוד השאלה: 4.00

נתונה קבוצה (**set**) אשר מייצגת אוסף של קטעים על הישר. כל קטע מיוצג על ידי זוג מספרים (tuple), כאשר הראשון אינו גדול מהשני.

השלימו את הקוד במקומות המסומנים, כך שהפונקציה **unify** (אשר אינה רקורסיבית) תאחד בין קטעים שיש ביניהם חפיפה לקטע אחד גדול יותר.

לאחד קטעים הכוונה היא להוציא את שני הקטעים המקוריים מהאוסף, ולהכניס במקומם קטע חדש אשר מכיל בדיוק את שני הקטעים, ולא יותר.

הפונקציה **product** מתוך הספריה **itertools** מייצרת איברים של מכפלה קרטזית (בשאלה הזו, זוגות של בחירת איבר מהקלט הראשון ואיבר מהקלט השני) בעבור הקלטים שלה. חיסור קבוצה (**set**) מקבוצה מוציאה אברים מקבוצה. השיטה **add** מוסיפה איבר בודד לתוך קבוצה.

```
import itertools as it
def unify(intervals):
    while True:
        for e1,e2 in it.product(intervals,intervals):
            if ______(1)______:
                continue
            if e1[0]<=e2[1] and e2[0]<=e1[1]:
                intervals-={______(2)______}
                intervals.add(______(3)______)
                ______(4)______
        else:
            return

intervals={(1,4),(3,6),(5,8),(9,10)}
unify(intervals)
assert intervals=={(1,8),(9,10)}
```

(1)

(2)

(3)

(4)

שאלה **11**

לא נענה

ניקוד השאלה: 15.00

במשחק דומינו, ישנם אבנים בצורה מלבנית אשר חצויים לאורכם, כאשר על כל חצי של מלבן ישנו מספר כלשהו של נקודות.
המטרה במשחק היא ליצור רצף של אבנים כך שכל אבן נוגעת בצד של אבן אחרת עם אותו מספר נקודות. נייצג אבנים בעזרת טאפלים (tuples), ורצף אבנים כרשימה של טופלים.

לדוגמה, הרצף **`[(3,5),(5, 6),(6,6),(6,2)]`** הוא רצף חוקי, ואילו הרצף **`[(4,1),(2,5),(5,6)]`** הוא רצף שאינו חוקי, מכיוון שהערכים (1) ו-(2) בשתי אבני הדומינו הראשונות, אינם זהים.

כתבו פונקציה **`find_path(domlist)`** אשר מקבלת רשימה של אבני דומינו, ומחזירה את הרצף החוקי הארוך ביותר שניתן למצוא ברשימה זו. במקרה ויש יותר מאחד, החזירו אחד מהם.

- ניתן להניח שמקבלים רשימה תקינה של אבני דומינו.
- לא ניתן להניח הנחות אחרות על הרשימה. בפרט, לא ניתן להניח שהרשימה לא ריקה.
- לא ניתן להפוך אבן דומינו.
- **אין** לשנות את רשימת הקלט.

לדוגמה:

```
>>> find_path([(5,3),(1,4),(4,5),(5,2),(3,8)])
[(1, 4), (4, 5), (5, 3), (3, 8)]
>>> find_path([(1,4),(2,3),(1,2)])
[(1, 2), (2, 3)]
>>> find_path([(100,200),(200,100),(100,200)])
[(100, 200), (200, 100), (100, 200)]
>>> find_path([(1,2)])
[(1, 2)]
>>> find_path([])
[]
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **12**

לא נענה

ניקוד השאלה: 15.00

נתונה המחלקה הבאה, המייצגת קודקוד בעץ בינארי (עץ שבו לכל קודקוד יש לכל היותר שני בנים):

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

נקרא לעץ בינארי **unival** (קרי, עץ "**חד-ערכי**") אם לכל הקדקדים בעץ (כולל לשורש) יש את אותו הערך. שימו לב שעץ עם קדקד אחד (עלה) הוא תמיד חד-ערכי.

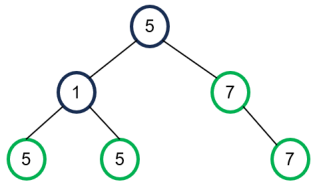
כתבו פונקציה **`count_unival_subtrees(root)`** המקבלת שורש של עץ בינארי (אוביקט מסוג **`TreeNode`**) ומחזירה את מספר הקודקדים בעץ שהם שורשים של עצים חד-ערכיים.

הערות:

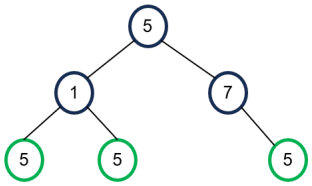
- אין לשנות את הקלט.
- אין לשנות את המחלקה הנתונה.
- אי אפשר להניח שמדובר בעץ מלא (קרי, עץ שלכל קודקוד שאינו עלה יש בדיוק 2 בנים)
- אי אפשר להניח שמדובר בעץ חיפוש בינארי

דוגמאות:

```
>>> root = TreeNode(5)
>>> root.left = TreeNode(1)
>>> root.right = TreeNode(7)
>>> root.left.left = TreeNode(5)
>>> root.left.right = TreeNode(5)
>>> root.right.right = TreeNode(7)
>>> count_unival_subtrees(root)
4
```




```
>>> root = TreeNode(5)
>>> root.left = TreeNode(1)
>>> root.right = TreeNode(7)
>>> root.left.left = TreeNode(5)
>>> root.left.right = TreeNode(5)
>>> root.right.right = TreeNode(5)
>>> count_unival_subtrees(root)
3
```



Answer:

(penalty regime: 0 %)

Reset answer

```
t=None, right=None):



IS LINE ###

:
```

שאלה **13**

לא נענה

ניקוד השאלה: 15.00

שימו לב שנוספו הסברים לחישוב מבצעים בתוך הדוגמאות

כתבו מימוש למערכת של עגלת קניות אינטרנטית. המערכת תורכב ממימוש של שתי מחלקות. מחלקה בשם **Item** המייצגת מוצרים, ומחלקה בשם **ShoppingCart** המייצגת עגלת קניות.

הבנאי של **Item** מקבל שלושה פרמטרים:

- **name** - שם המוצר (מחרוזת)
- **price** - מחיר סטנדרטי של המוצר (מספר שלם וחיובי)
- **sale_price** - מחיר מבצע (טאפל (tuple) של כמות ומחיר). ניתן להניח שהערכים חיוביים, ושמחיר המבצע אינו גבוה מהמחיר הרגיל. אם אין מבצע, הערך יהיה **None**.

יש לשמור את כל המוצרים הקיימים במשתנה מחלקה, כך שיהיה ניתן לקבל אוביקט של מוצר לפי שמו. יצירת אוביקט חדש עם שם קיים תחליף את האוביקט הקודם.

נתון השלד הבא של מחלקת המוצרים. אין לשנות את חתימת הבנאי, אך ניתן לשנות את שאר הקוד כפי הצורך. בפרט, ניתן לשנות את שמות השדות, להוסיף ולהוריד שדות, ולהוסיף שיטות חדשות.

```
class Item:
    all_items = ...

    def __init__(self, name, price, sale_price=None):
        self.name=name
        self.price=price
        self.sale_price=sale_price
        ...
```

יש לממש את השיטות הבאות במחלקה **ShoppingCart**:

- **__init__** - מייצר עגלת קניות ריקה.
- **add_item(self, name:str, quantity:int)** - מוסיף לעגלת הקניות כמות מסוימת של מוצר אשר מזוהה לפי שמו. אם לא קיים מוצר בשם זה, או שהמוצר כבר נמצא בעגלה, יש להעלות שגיאת **ValueError**.
- **remove_item(self, name:str)** - מוציא מוצר מעגלת הקניות. אם המוצר לא נמצא בעגלה, יש להעלות **ValueError**.
- **get_total(self)** - מחזיר את מחיר המוצרים שבעגלה, כולל מבצעים.
- **get_savings(self)** - מחזיר את הסכום שנחסך בקניה. הסכום שנחסך הוא ההפרש בין המחיר הרגיל של כל המוצרים והמחיר כולל המבצעים.
- **get_list(self)** - מחזיר את רשימת הקניות, כרשימה של זוגות של שם מוצר וכמות. סדר האברים ברשימה לא משנה.

דוגמאות:

```
>>> Item("Apple",4)
<...Item...>
>>> Item("Tomato",5,(2,7))
<...Item...>
>>> cart=ShoppingCart()
>>> cart.add_item("Apple", 5)
>>> cart.add_item("Tomato", 4)
>>> cart.get_total()
34 # 5*4+2*7     # (4/2=2) חמשה תפוחים, וכל תפוח בארבע. כל שתי עגבניות בשבע
>>> cart.add_item("Carrot",6) # Doesn't exist
...
ValueError:...
>>> cart.add_item("Tomato",7) # Already in cart
...
ValueError:...
>>> cart.remove_item("Tomato")
>>> cart.add_item("Tomato",7)
>>> cart.get_total()
46 # 5*4+(3*7+1*5)     # (שלא נכנסת למבצע) והעגבניה האחרונה (שלש זוגות) חמשה תפוחים, וכל תפוח בארבע. כל שתי עגבניות בשבע
בחמש
>>> cart.get_savings()
9  # (5*4+7*5)-46
>>> cart.get_list()
[('Apple', 5), ('Tomato', 7)]
```

Answer:

(penalty regime: 0 %)

Reset answer

```
ice, sale_price=None):

ice




, quantity:int):

str):
```

שאלה **14**

לא נענה

ניקוד השאלה: 15.00

נתונה המחלקה הבאה, המייצגת קודקוד בעץ בינארי (עץ שבו לכל קודקוד יש לכל היותר שני בנים):

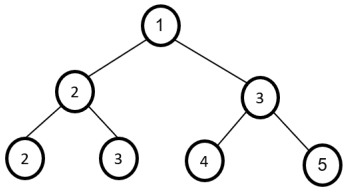
```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

עץ בינארי ייקרא **prefix search tree** (ובעברית: "**עץ חיפוש קדימי**") אם מתקיימים כל התנאים הבאים:

- הערך המינימלי בעץ נמצא בשורש
- כל ערך בענף השמאלי קטן או שוה מכל ערך בענף הימני
- שני הענפים (במידה והם קיימים) הם גם עצי חיפוש קדימיים.

שימו לב שעץ עם קדקד אחד (עלה) הוא תמיד עץ חיפוש קדימי.

דוגמא לעץ חיפוש קדימי:



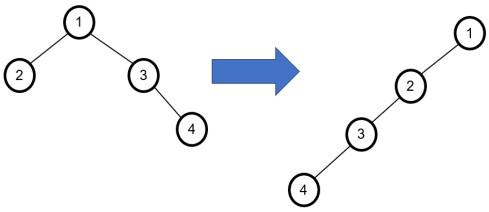
כתבו פונקציה **`unbalance_tree(root)`** המקבלת שורש של עץ חיפוש קדימי (אוביקט מסוג **`TreeNode`**). הפונקציה תשנה את העץ כך שיהיה עץ חיפוש קדימי לא מאוזן, שבו לכל קדקד שאינו עלה יהיה רק בן שמאלי, ולכל העץ יהיה עלה אחד בלבד.

<u>הערות</u>:

- הפונקציה אינה צריכה להחזיר ערך. שורש העץ הישן יהיה גם השורש של העץ החדש.
- אין לשנות את המחלקה הנתונה.
- אין לייצר קודקדים חדשים, או לשנות את הערכים של הקודקדים הקיימים. עם זאת, אפשר (וצריך) לשנות את הקישורים שבין הקדקדים.

<u>לדוגמא</u>:

```
>>> root = TreeNode(1)
>>> root.left = TreeNode(2)
>>> root.right = TreeNode(3)
>>> root.right.right = TreeNode(4)
>>> unbalance_tree(root)
>>> root.val
1
>>> root.left.val
2
>>> root.left.left.val
3
>>> root.left.left.left.val
4
```



Answer:

(penalty regime: 0 %)

Reset answer

```
t=None, right=None):



IS LINE ###

e:
```

שאלה **15**

לא נענה

ניקוד השאלה: 15.00

כתבו פונקציה **palindrome_partition(s)** המקבלת מחרוזת **s**, ומחזירה רשימה של רשימות המכילות כל חלוקה (**partition**) אפשרית של המחרוזת לפלינדרומים שאינם ריקים.
נזכיר שפלינדרום הוא מחרוזת שניתן לקרוא באותו אופן משני הקצוות (למשל, 'c', 'aba', וכו'.)

הערות:

- סדר החלוקות בתוצאה המוחזרת אינו חשוב, אך סדר המחרוזות בתוך כל חלוקה, הכרחי.
- מחרוזת ריקה היא פלינדרום.
- שימו לב שלכל מחרוזת קיימת לפחות חלוקה חוקית אחת.

לדוגמא:

```
>>> palindrome_partition('aab')
[['a', 'a', 'b'], ['aa', 'b']]

>>> palindrome_partition('aabcb')
[['a', 'a', 'b', 'c', 'b'], ['aa', 'b', 'c', 'b'], ['a', 'a', 'bcb'], ['aa', 'bcb']]

>>> palindrome_partition('aba')
[['a', 'b', 'a'], ['aba']]

>>> palindrome_partition('c')
[['c']]

>>> palindrome_partition('')
[[]]
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **16**

לא נענה

ניקוד השאלה: 15.00

נתונה המחלקה הבאה המאפשרת לחקור ביעילות אהדה ברשתות חברתיות:

```
class SocialRanking:
    def __init__(self, user_likes):
        self.user_likes = {}
        self.user_rank = {}
        self.rank_user = {}
        ranked_list=sorted(user_likes.items(),key=lambda x:-x[1])
        for rank,(user,likes) in enumerate(ranked_list,1):
            self.user_likes[user] = likes
            self.user_rank[user] = rank
            self.rank_user[rank] = user
```

אבייקטים מהמחלקה מכילים את המילונים הבאים:

- **user_likes** - ממפה שמות משתמשים למספר הלייקים שיש לאותו המשתמש.
- **user_rank** - ממפה משתמשים לדירוג של המשתמש מתוך כלל המשתמשים. המשתמש הכי פופולרי ידורג 1. במקרה של שוויון, אחד המשתמשים ידורג לפני השני באופן שרירותי. המילון מאפשר לברר באופן יעיל מה דירוגו של משתמש כלשהו.
- **rank_user** - מילון הפוך למילון הקודם. המילון האפשר לברר באופן יעיל מי המשתמש שמדורג במקום ה **k**. גם כאן הדירוג בין משתמשים עם אותו מספר לייקים הוא שרירותי, אך חייב להיות עקבי עם מילון הדירוגים הקודם.

יש להוסיף למחלקה את השיטה (מתודה) הבאה:

update(self,user,likes) - השיטה תעדכן את מספר הלייקים של המשתמש לערך החדש, ותתקן (במידת הצורך) גם את מילוני הדירוגים כדי שיתאימו למידע המעודכן שהוכנס לתוכו. אם המשתמש לא נמצא באוביקט, השיטה תוסיף אותו.

הערות:

- יש להניח שהשינויים בדירוגים בעקבות שינוי מספר הלייקים של משתמש יהיו מאד קטנים ביחס למספר המשתמשים הכולל, וכן שמשתמש חדש ידורג קרוב לתחתית הדירוג.
- ניתן להניח שמספר הלייקים הוא אי שלילי, ושכל הקלטים תקינים.
- יתכן שעדכון יקטין את מספר הלייקים של משתמש.
- סדר הדירוג לא משנה אם מספר הלייקים שוה, אך יש לשמור על המילונים עקביים.
- אין להוסיף שדות למחלקה ואין לשנות את הבנאי. ניתן להוסיף שיטות חדשות למחלקה, וניתן להוסיף פונקציות מחוץ למחלקה.

דוגמאות:

```
>>> ranking = SocialRanking({'R':130,'G':120,'W':10})
>>> ranking.user_likes
{'R':130, 'G':120, 'W':10}
>>> ranking.user_rank
{'R':1, 'G':2, 'W':3}
>>> ranking.rank_user
{1:'R', 2:'G', 3:'W'}

>>> ranking.update('Y',110)
>>> ranking.user_likes
{'R':130, 'G':120, 'W':10, 'Y':110}
>>> ranking.user_rank
{'R':1, 'G':2, 'W':4, 'Y':3}
>>> ranking.rank_user
{1:'R', 2:'G', 3:'Y', 4:'W'}

>>> ranking.update('G',140)
>>> ranking.user_likes
{'R':130, 'G':140, 'W':10, 'Y':110}
>>> ranking.user_rank
{'R':2, 'G':1, 'W':4, 'Y':3}
>>> ranking.rank_user
{1:'G', 2:'R', 3:'Y', 4:'W'}
```

Answer:

(penalty regime: 0 %)

Reset answer

```
es):



_likes.items(),key=lambda x:-x[1])
n enumerate(ranked_list,1):
r] = likes
] = rank
```

```
] = user

E THIS LINE ###

:
```

שאלה **17**

לא נענה

ניקוד השאלה: 1.00

סמנו איזה שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

- שאלה 11 - דומינו
- שאלה 12 - עץ חד ערכי
- שאלה 13 - עגלת קניות
- שאלה 14 - עץ לא מאוזן
- שאלה 15 - פלינדרומים
- שאלה 16 - דירוג חברתי

שאלה **18**

לא נענה

לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.